

CANDIDATE ASSESSMENT DOCUMENT

Medicclaim Pro

Healthcare Claims & Policy Management Platform

Project Title	MediClaim Pro — Healthcare Claims & Policy Management Platform
Target Level	Mid-Senior to Senior .NET Developer
Estimated Duration	3–4 Working Days
Technology Stack	C# 12 / .NET 8 / ASP.NET Core / EF Core 8 / SQL Server
Architecture	Clean Architecture + CQRS + MediatR + Hangfire
Classification	Confidential — Candidate Use Only

This document is the complete specification for your technical assessment.
Read all sections before writing a single line of code.

1. Project Overview

1.1 Business Scenario

MediClaim Pro is a multi-tenant SaaS platform used by healthcare insurance companies to manage patient policies, process medical claims, schedule provider appointments, and track fraud indicators. Each insurance company (called a tenant) operates in a fully isolated data environment with zero cross-tenant visibility.

The platform must handle high claim volumes, enforce strict business rules around policy coverage and claim eligibility, generate automated notifications when claims change state, produce regulatory audit reports, and flag suspicious claim patterns for manual review.

What this assessment tests

Your ability to design and implement a production-grade, enterprise backend in C# / .NET 8.

Architecture discipline: Clean Architecture, CQRS, and dependency inversion under real business pressure.

Business logic complexity: stateful workflows, concurrency safety, and fraud detection rules.

Security: JWT with refresh token family strategy, multi-tenant data isolation, and RBAC.

Database engineering: SQL Server schema design, optimistic concurrency, stored procedures, and EF Core.

Testing: unit tests for domain logic, integration tests for end-to-end flows, and concurrency scenarios.

1.2 Platform Actors & Roles

Role	Responsibilities & Permissions
SuperAdmin	Manages tenants (insurance companies). Activates / suspends tenants. Views cross-tenant health metrics (no claim data). Manages system-wide configuration.
TenantAdmin	Manages users within their tenant. Configures policy types and coverage rules. Views all claims and audit reports. Approves or rejects escalated claims. Unlocks locked accounts.
ClaimsOfficer	Reviews and processes claims assigned to them. Requests additional documents from patients. Approves claims up to their configured approval limit. Cannot override fraud-flagged claims without TenantAdmin approval.
Patient	Submits new claims against active policies. Uploads supporting documents. Tracks claim status. Books appointments with in-network providers.

1.3 Core Claim Lifecycle

State Machine — All Transitions Must Be Enforced in Code

Draft → Submitted → UnderReview → PendingDocuments → Escalated
→ Approved → Settled
→ Rejected
→ Withdrawn (patient withdraws)

Rules:

- Invalid transitions must return HTTP 422 Unprocessable Entity with a descriptive reason.
- The transition logic must live in the domain entity as a state-machine method, not in a controller or handler.
- Every transition writes an immutable entry to AuditLogs and ClaimStatusHistory.
- A Settled or Rejected claim cannot be re-opened — terminal states.

2. Functional Requirements

FR-01 Multi-Tenant Identity & Access

- Tenant registration with subdomain slug (e.g. acme.mediclaim.io) — slug is immutable after creation
- JWT access tokens (15-min TTL) + refresh tokens (7-day TTL) stored hashed in DB, one active refresh per device
- Refresh token rotation: old token invalidated on each refresh; detect reuse and revoke all sessions (token family strategy)
- Password policy: min 8 chars, 1 uppercase, 1 digit, 1 special character — enforced server-side via FluentValidation
- Account lockout after 5 consecutive failed login attempts; unlockable by TenantAdmin only
- All tokens carry TenantId and UserId claims; middleware validates tenant is active on every request

FR-02 Policy Management

- Policy types defined by TenantAdmin: name, description, premium, coverage categories, max annual limit
- Patient enrolls in a policy — generates a unique policy number: POL-{YEAR}-{TenantCode}-{SEQ}
- Policy has an active date range; claims can only be submitted against active policies
- Annual benefit limit tracked in real time; submitting a claim that would exceed remaining balance is blocked
- Mid-year policy upgrades allowed — coverage delta calculated from upgrade date, not policy start date
- Policy renewal generates a new policy record; prior claims are archived and limits reset

FR-03 Claims Processing

- Patient creates a claim draft referencing: policy, treatment date, ICD-10 diagnosis code, provider, and amount
- On submission, system auto-validates: policy is active, balance is sufficient, treatment is within coverage categories
- ClaimsOfficer assigned automatically via round-robin load balancing within the tenant
- Officers approve claims up to their configured approval limit; above that limit auto-escalates to TenantAdmin
- Claim settlement deducts from policy annual balance atomically (SQL transaction + optimistic concurrency)
- Duplicate claim detection: same patient + provider + ICD-10 code + treatment date → block and return reason

FR-04 Appointment & Provider Management

- Providers registered with specialties, in-network status per tenant, and slot availability templates
- Patient books appointment against a claim (optional link) or standalone — enforces no-double-booking per provider
- Appointment states: Requested → Confirmed → Completed | Cancelled | NoShow
- Cancellation policy: free cancellation > 24 h before; within 24 h is flagged as NoShow-risk
- Provider availability managed via weekly recurring schedule templates + one-off date overrides

FR-05 Audit Logging & Activity Tracking

- Every state change on Claim, Policy, User, and Appointment writes an immutable audit record
- Audit record must capture: entity type, entity ID, old state (JSON), new state (JSON), changed by, timestamp (UTC), IP address, correlation ID
- Audit records are INSERT ONLY — no UPDATE or DELETE operations allowed on the AuditLogs table
- TenantAdmin can query audit history for any entity with date-range filtering and pagination
- Audit export endpoint produces a signed CSV download for regulatory submission

FR-06 Notifications & Background Jobs

- Email notification on every claim state transition (template-driven, per-tenant branding; mock SMTP is acceptable)
- Daily job at 06:00 UTC: scan claims in PendingDocuments state for > 7 days — auto-reject with reason code
- Weekly job: generate a claims summary report per tenant and email it to TenantAdmin
- Appointment reminder: send notification 24 h before scheduled appointment start time
- Job execution history logged to JobExecutionLogs table: JobName, StartedAt, FinishedAt, Status, RecordsAffected, ErrorMessage

3. Non-Functional Requirements

Requirement	Expectation
Clean Architecture	Four projects: Domain, Application, Infrastructure, API. Domain has zero external dependencies. Application depends only on Domain. All dependency arrows point inward.
CQRS + MediatR	All reads via Query handlers. All writes via Command handlers. Pipeline behaviours handle cross-cutting concerns.
Async / Await	All I/O operations must be fully async. No .Result, .Wait(), or Thread.Sleep() anywhere in the main code path.
Performance	All list endpoints paginated (cursor-based preferred). No unbounded SELECT * queries. EF projections to DTOs — never load full entity graphs for reads.
Structured Logging	Serilog to Console + File sink. Every log entry carries CorrelationId, TenantId, and UserId where available.
Error Handling	Custom exception hierarchy mapped to HTTP status codes. RFC 7807 ProblemDetails format on all error responses. Stack traces never exposed to clients.
Concurrency	EF Core RowVersion concurrency tokens on Policies and Claims. DbUpdateConcurrencyException caught and mapped to HTTP 409 Conflict.
Security	No raw SQL except for the specified stored procedure. No secrets in source code. Sensitive fields (SSN, DOB) encrypted at rest using AES-256.
API Documentation	Swagger / OpenAPI 3.0 with XML comments, example bodies, and Bearer auth scheme configured.
Testing	Unit tests: Domain + Application layer. Integration tests: full HTTP pipeline via WebApplicationFactory. Coverage ≥ 80% on Domain + Application layers.

4. Database Design

4.1 SQL Server Schema — Required Tables

Design and implement the following tables as code-first EF Core entities. All tables must include CreatedAt, UpdatedAt, and IsDeleted columns enforced via a SaveChangesInterceptor.

Tenants			
TenantId	UNIQUEIDENTIFIER	PK	Default: newsequentialid()
Slug	NVARCHAR(50)	UNIQUE	Immutable after creation

Name	NVARCHAR(200)		
Status	TINYINT	IDX	Enum: Active / Suspended / Trial
Settings	NVARCHAR(MAX)		JSON: per-tenant config (CORS, branding, limits)
CreatedAt / UpdatedAt	DATETIME2		UTC always

Users			
UserId	UNIQUEIDENTIFIER	PK	
TenantId	UNIQUEIDENTIFIER	FK	Indexed; tenant-scoped
Email	NVARCHAR(256)	IDX	Unique within tenant, not globally
PasswordHash	NVARCHAR(512)		BCrypt hash
Role	TINYINT		Enum: SuperAdmin / TenantAdmin / ClaimsOfficer / Patient
ApprovalLimit	DECIMAL(18,2)		Nullable; applies to ClaimsOfficer
IsFraudSpecialist	BIT		For round-robin assignment routing
IsOnLeave	BIT		Excludes officer from assignment queue
FailedLoginCount	INT		Reset on successful login
IsLocked / LockedAt	BIT / DATETIME2		Account lockout support
SsnEncrypted	VARBINARY(256)		AES-256; decrypt only via EncryptionService

Policies			
PolicyId	UNIQUEIDENTIFIER	PK	
TenantId	UNIQUEIDENTIFIER	FK	Indexed
PatientId	UNIQUEIDENTIFIER	FK	→ Users
PolicyTypeId	UNIQUEIDENTIFIER	FK	→ PolicyTypes
PolicyNumber	NVARCHAR(30)	UNIQUE	Format: POL-{YEAR}-{TenantCode}-{SEQ}
AnnualLimit	DECIMAL(18,2)		Total coverage amount
UsedAmount	DECIMAL(18,2)		Updated on settlement; protected by RowVersion

StartDate / EndDate	DATE		Active coverage window
RowVersion	ROWVERSION		Optimistic concurrency token

Claims			
ClaimId	UNIQUEIDENTIFIER	PK	
TenantId	UNIQUEIDENTIFIER	FK	Indexed
PolicyId / PatientId	UNIQUEIDENTIFIER	FK	Both indexed
AssignedOfficerId	UNIQUEIDENTIFIER	FK	→ Users; nullable until assigned
ClaimNumber	NVARCHAR(30)	UNIQUE	Auto-generated on submission
ICD10Code	NVARCHAR(10)	IDX	Must match regex pattern server-side
ProviderId	UNIQUEIDENTIFIER	FK	→ Providers; indexed
ClaimedAmount / ApprovedAmount	DECIMAL(18,2)		ApprovedAmount nullable until reviewed
Status	TINYINT	IDX	Enum: Draft/Submitted/UnderReview/PendingDocuments/Escalated/Approved/Settled
FraudScore	DECIMAL(5,2)		0.00–100.00; computed asynchronously post-submission
TreatmentDate	DATE		Must not be in the future
SubmittedAt / ResolvedAt	DATETIME2		UTC; nullable until state reached

AuditLogs			
AuditId	BIGINT IDENTITY	PK	Sequential integer; not GUID
TenantId	UNIQUEIDENTIFIER	IDX	For tenant-scoped queries
EntityType	NVARCHAR(50)		e.g. 'Claim', 'Policy'
EntityId	NVARCHAR(36)	IDX	GUID as string for flexibility
Action	NVARCHAR(50)		Created / StatusChanged / Updated / Deleted
OldValues	NVARCHAR(MAX)		JSON snapshot of previous state
NewValues	NVARCHAR(MAX)		JSON snapshot of new state
ChangedByUserId	UNIQUEIDENTIFIER		No FK — intentional; user may be deleted

IpAddress	NVARCHAR(45)		IPv4 or IPv6
CorrelationId	NVARCHAR(36)		Matches X-Correlation-ID header
Timestamp	DATETIME2	IDX	UTC; indexed for range queries

RefreshTokens			
TokenId	UNIQUEIDENTIFIER	PK	
UserId	UNIQUEIDENTIFIER	FK	Indexed
FamilyId	UNIQUEIDENTIFIER	IDX	All tokens in a rotation chain share this ID
TokenHash	NVARCHAR(512)	UNIQUE	SHA-256 hash of raw token
IsUsed	BIT		True after first rotation; used = stolen if presented again
IsRevoked	BIT		True when family is invalidated
DeviceInfo	NVARCHAR(200)		User-Agent substring for display
CreatedAt / ExpiresAt	DATETIME2		UTC

4.2 Database Engineering Constraints

- Composite UNIQUE index on Claims(TenantId, PatientId, ProviderId, ICD10Code, TreatmentDate) — duplicate claim detection
- Filtered index on Claims(AssignedOfficerId, Status) WHERE Status IN (2,3,4) — optimises officer workload queries
- All FK columns must be explicitly indexed — do not rely on EF Core default behaviour
- Use DECIMAL(18,2) for all monetary values — never FLOAT or MONEY
- AuditLogs table uses IDENTITY BIGINT PK — append-only, no EF change tracking on this table
- A stored procedure is required for claim settlement: deducts from policy balance and updates claim status atomically
- A SQL View is required for officer workload summary, consumed by the reporting endpoint
- All migrations must be code-first; no raw SQL migration scripts accepted

5. Mandatory API Endpoints

5.1 Authentication — /api/auth

Method	Endpoint	Role	Description
POST	/api/auth/register	Public	Tenant + first TenantAdmin registration; idempotent on email

POST	/api/auth/login	Public	Returns access + refresh token pair; logs IP and device
POST	/api/auth/refresh	Public	Rotates refresh token; detects reuse → revoke entire family
POST	/api/auth/revoke	Authenticated	Logs out — invalidates current refresh token
POST	/api/auth/unlock/{userId}	TenantAdmin	Unlocks a locked account; logs the unlock action

5.2 Policies — /api/policies

Method	Endpoint	Role	Description
GET	/api/policies/types	TenantAdmin, Patient	List all policy types for current tenant
POST	/api/policies/types	TenantAdmin	Create policy type with coverage categories and limits
POST	/api/policies/enroll	Patient	Enroll in a policy; generate unique policy number atomically
GET	/api/policies/{policyId}	Patient (own), TenantAdmin	Detail with remaining balance and claim history
PUT	/api/policies/{policyId}/upgrade	Patient, TenantAdmin	Upgrade policy type; recalculate remaining limit
POST	/api/policies/{policyId}/renew	TenantAdmin	Renew policy; archive prior record; reset limits

5.3 Claims — /api/claims

Method	Endpoint	Role	Description
POST	/api/claims/draft	Patient	Create draft; runs all business rule pre-validations
POST	/api/claims/{claimId}/submit	Patient	Submit draft; triggers fraud scoring + officer assignment
PATCH	/api/claims/{claimId}/transition	ClaimsOfficer, TenantAdmin	Move to next valid state; 422 on invalid transition
POST	/api/claims/{claimId}/approve	ClaimsOfficer, TenantAdmin	Approve with approved amount; enforces officer limit
POST	/api/claims/{claimId}/settle	TenantAdmin	Execute settlement SP; deduct from policy balance
POST	/api/claims/{claimId}/reject	ClaimsOfficer, TenantAdmin	Reject with mandatory reason code

POST	/api/claims/{claimId}/documents	Patient	Upload supporting documents (multipart/form-data)
GET	/api/claims/my	Patient	Patient's own claims — cursor paginated
GET	/api/claims/officer/queue	ClaimsOfficer	Assigned open claims sorted by fraud score desc
GET	/api/claims/fraud-flags	TenantAdmin	All claims with FraudScore $\geq$ configurable threshold

5.4 Appointments — /api/appointments

Method	Endpoint	Role	Description
GET	/api/appointments/providers/available	Patient	Available slots for specialty + date range
POST	/api/appointments/book	Patient	Book slot; double-booking guard at DB transaction level
POST	/api/appointments/{id}/cancel	Patient, TenantAdmin	Cancel; applies late-cancellation NoShow policy

5.5 Reports — /api/reports

Method	Endpoint	Role	Description
GET	/api/reports/claims-summary	TenantAdmin	Monthly aggregated claims with approval rates
GET	/api/reports/officer-workload	TenantAdmin	Officer performance metrics from SQL View
GET	/api/reports/audit-export	TenantAdmin	Signed CSV export of audit logs for date range

6. Architecture Expectations

6.1 Clean Architecture — Solution Structure

Project Layout — Mandatory	
MediClaim.Domain	— Entities, value objects, domain events, enums, custom exceptions. ZERO external NuGet dependencies.
MediClaim.Application	— CQRS Commands and Queries (MediatR), FluentValidation validators, interface contracts (IClaimRepository, IFraudScoringService, etc.),

DTOs and mapping profiles.
Depends on Domain ONLY.

MediClaim.Infrastructure — EF Core DbContext + Migrations, Repository implementations, Hangfire job implementations, Email service (mock acceptable), Encryption service, File storage service.
Depends on Application (implements its interfaces).

MediClaim.API — Controllers, Middleware pipeline, DI wiring, Swagger config, Program.cs entry point.
Depends on Application; references Infrastructure ONLY for DI.

MediClaim.Tests.Unit — Domain + Application handler tests. No EF. No HTTP.

MediClaim.Tests.Integration — WebApplicationFactory tests with SQL Server LocalDB or TestContainers.

Dependency Rule — Enforced During Review

VIOLATION: Application project references MediClaim.Infrastructure → Automatic architectural defect

VIOLATION: Domain project has any NuGet dependency except

Microsoft.Extensions.Logging.Abstractions

VIOLATION: Controller methods contain business logic beyond request parsing and response mapping

VIOLATION: DbContext accessed directly from a Controller or Application handler

6.2 CQRS Pipeline Behaviours — Order Matters

Behaviour	What It Must Do
CorrelationIdBehaviour	Attaches correlation ID from HttpContext to all Serilog log scopes for this request.
TenantValidationBehaviour	Verifies tenant is in Active status before any handler executes. Returns 403 if suspended.
ValidationBehaviour<T>	Runs all FluentValidation validators for the command/query. Returns structured ProblemDetails with field-level errors on failure.
AuthorizationBehaviour<T>	Checks resource-level ownership. A Patient can only access their own claims/policies — not just any authenticated user.
PerformanceBehaviour<T>	Logs a Warning via Serilog if a handler takes longer than 500 ms, including handler name and tenant.
AuditBehaviour<T>	For commands implementing IAuditableCommand, writes an AuditLog record after successful execution.

6.3 EF Core Conventions

- Global query filter on TenantId and IsDeleted applied in OnModelCreating — no accidental cross-tenant data access
- SaveChangesInterceptor auto-sets CreatedAt, UpdatedAt, and soft-delete stamp on every save
- AuditLog table written via DbContext.Database.ExecuteSqlAsync — never EF-tracked
- All monetary columns use HasPrecision(18, 2) enforced via a convention in OnModelCreating
- Repository pattern: IRepository<T> base + specific interfaces for domain query methods
- Unit of Work wraps repositories — single SaveChangesAsync call per request in the command handler

6.4 Error Handling Strategy

- Custom exception hierarchy: DomainException, ValidationException, NotFoundException, ConflictException, ForbiddenException
- Global exception middleware maps each type: DomainException → 422, NotFoundException → 404, ConflictException → 409, ForbiddenException → 403, unhandled → 500
- All error responses use RFC 7807 ProblemDetails format with a traceId field matching the X-Correlation-ID header
- HTTP 500 errors log the full stack trace via Serilog at Error level; 4xx errors log at Warning level
- Stack traces and internal exception messages are NEVER included in the response body

7. Middleware Requirements

All five middleware components below are mandatory. Implement each as a separate class inheriting from IMiddleware or using the convention-based approach.

Middleware	Specification
MW-01 TenantResolutionMiddleware	Extracts TenantId from JWT claim tenant_id on every authenticated request. Loads TenantContext from Redis cache or DB. Populates ICurrentTenantService scoped instance. Returns HTTP 403 if tenant is suspended; 401 if tenant not found. Skips for /api/auth/register and /api/auth/login only.
MW-02 CorrelationIdMiddleware	Reads X-Correlation-ID header from incoming request, or generates a new GUID if absent. Pushes into Serilog LogContext and HttpContext.Items. Adds X-Correlation-ID to every response header. Must run first — before all other middleware — so every log entry carries the ID.
MW-03 RequestTimingMiddleware	Measures wall-clock time for the full request/response cycle. Appends X-Response-Time-Ms to every response header. Logs any request exceeding 1000 ms at Warning level with route, HTTP method, status code, and TenantId.
MW-04 RateLimitingMiddleware (custom)	Sliding window counter keyed by IP + route, stored in memory (or Redis). Limit: 10 requests per 60-second window on auth endpoints. Returns HTTP 429 Too Many Requests with a Retry-After header. You must implement the

	sliding window algorithm yourself — do NOT use <code>Microsoft.AspNetCore.RateLimiting</code> .
MW-05 AuditRequestMiddleware	Buffers response body for all mutating requests (POST, PUT, PATCH, DELETE). After the handler completes, stores a <code>RequestAuditContext</code> in <code>HttpContext.Items</code> containing the original request snapshot and response. Used by <code>AuditBehaviour</code> downstream. Must NOT buffer GET responses.

8. Scenarios

All six scenarios are mandatory. Partial implementation must be documented in your README.

Scenario 1 — Fraud Scoring Engine

On every claim submission, compute a `FraudScore` (0–100) using the weighted rule set below. The score is computed asynchronously, but it must be persisted before the claim becomes visible to officers.

Scoring Rules (additive):

- +25 pts — Same patient submitted > 3 claims in 30 days to different providers with the same ICD-10
- +20 pts — Claimed amount exceeds 150% of the median approved amount for that ICD-10 within the tenant
- +20 pts — Treatment date is a weekend or public holiday AND provider specialty is non-emergency
- +15 pts — Patient had a claim rejected for the same ICD-10 code in the past 90 days
- +10 pts — Provider has a rejection rate above 40% in the past 6 months within this tenant
- +10 pts — Claim submitted less than 24 hours after the patient enrolled in the policy

Behaviour Rules:

- Claims with `FraudScore` ≥ 60 are auto-flagged: `ClaimsOfficer` cannot approve without explicit `TenantAdmin` override
- Score must be re-evaluated if any claim field is edited while the claim is in `Draft` or `PendingDocuments` state
- Implement as `IFraudScoringService` with each rule as a discrete, individually-testable evaluator

Scenario 2 — Round-Robin Officer Assignment with Workload Balancing

When a claim is submitted, assign a `ClaimsOfficer` within the tenant using the following algorithm:

1. Build eligible pool: active officers (`IsActive` = true, `IsOnLeave` = false) within the tenant
2. Primary sort: officer with the fewest open claims (status `UnderReview` or `PendingDocuments`)
3. Tie-break: officer with the oldest `LastAssignedAt` timestamp
4. If `FraudScore` ≥ 60 : only officers with `IsFraudSpecialist` = true are eligible
5. If no eligible officer exists: move claim to `PendingAssignment` queue and alert `TenantAdmin`

Concurrency Constraint:

- Assignment must be atomic — two concurrent claim submissions must not assign to the same officer if doing so would violate the load balancing rule.
- Use a SELECT ... WITH (UPDLOCK) hint or row-level lock inside an explicit transaction.

Scenario 3 — Policy Balance Deduction with Concurrency Safety

Settling a claim deducts the ApprovedAmount from the Policy.UsedAmount.

Two concurrent settlements against the same policy must not both succeed if the combined amount exceeds AnnualLimit.

Required implementation:

1. Stored procedure: usp_SettleClaim(@ClaimId, @ApprovedAmount, @PolicyId)
 - Re-checks available balance INSIDE a serialisable transaction before deducting
 - If balance insufficient: returns result code; application maps to HTTP 409 Conflict
 - Updates Claim.Status = Settled and Policy.UsedAmount in the same transaction
2. EF Core RowVersion token on Policy as a secondary guard before calling the SP
3. Settlement is idempotent: calling it twice with the same ClaimId returns the existing result, not a double deduction
4. Write an integration test that explicitly creates this race condition with two parallel HTTP calls and asserts exactly one succeeds (200) and one fails (409).

Scenario 4 — Mid-Year Policy Upgrade Coverage Recalculation

A patient upgrades from Policy Type A (limit £5,000) to Policy Type B (limit £8,000) mid-year.

The system must recalculate the remaining annual limit correctly.

Formula:

RemainingDays = (EndDate - UpgradeDate).TotalDays

NewAnnualLimit = (OldTypeLimit × (RemainingDays / 365)) + (NewTypeLimitDelta × (RemainingDays / 365))

Where: NewTypeLimitDelta = NewTypeLimit - OldTypeLimit

Example:

Old limit: £5,000. Already used: £3,000. Remaining: £2,000.

Upgrade on day 180 of a 365-day policy. Remaining days = 185.

Delta = £8,000 - £5,000 = £3,000

Pro-rated delta = £3,000 × (185/365) = £1,520.55

New remaining limit = £2,000 + £1,520.55 = £3,520.55

Constraints:

- Upgrade CANNOT reduce the remaining limit below the total of pending (Submitted/UnderReview) claims
- Audit record must capture before/after limit AND the formula inputs for regulatory traceability

- Unit test required with at least 3 edge cases (upgrade on day 1, upgrade on day 364, upgrade when fully used)

Scenario 5 — Refresh Token Reuse Detection (Token Family Strategy)

Implement the Refresh Token Family strategy to detect stolen token usage.

1. Each token belongs to a family (FamilyId GUID shared across all rotations in a session)
2. On token refresh: mark the presented token as IsUsed = true, issue a new token in the same family
3. If a token where IsUsed = true is presented again:
 - a. It was already rotated — this indicates a potential token theft
 - b. Immediately revoke ALL tokens in that family (IsRevoked = true)
 - c. Terminate all active sessions for that user across all devices
 - d. Queue a security-alert email to the user (async background job)
 - e. Log the suspicious event: UserId, IP address, timestamp → to AuditLogs
 - f. Return HTTP 401 with body: { error: 'Token reuse detected. All sessions revoked.' }
4. RefreshTokens table schema must support this — design it as part of your domain

Scenario 6 — Appointment Double-Booking Prevention Under Concurrency

Two patients attempt to book the exact same provider slot simultaneously.
Only one must succeed. The solution must be enforced at the DATABASE level.

1. Use SELECT ... WITH (UPDLOCK, ROWLOCK) on the relevant appointment slot row inside an explicit transaction (raw EF SQL or a stored procedure)
2. If the slot is already taken when the lock is acquired: the transaction rolls back
3. The losing request receives HTTP 409 Conflict
4. The 409 response body must include available alternative slots for the same provider on the same day (query and return up to 5 alternatives in the same response)
5. No application-level locking (no static locks, SemaphoreSlim, etc.)
— the guard must live entirely in the database transaction

Write an integration test that fires two concurrent POST /api/appointments/book requests for the same slot and asserts exactly one 200 and one 409.

9. Background Processing & Scheduled Jobs

Implement using Hangfire (preferred) or IHostedService. All jobs must honour CancellationToken and log their execution to the JobExecutionLogs table.

Job Name + Schedule	Specification
StaleDocumentAutoRejectJob Daily at 06:00 UTC	Query all claims in PendingDocuments status where UpdatedAt < Now - 7 days. For each: transition to Rejected (reason code: STALE_DOCUMENTS), write ClaimStatusHistory entry, notify patient. Must be idempotent — re-running must not double-reject.
WeeklyClaimsSummaryJob Every Sunday 07:00 UTC	For each active tenant: aggregate total claims, approved count, rejection rate, average processing time. Email formatted report to TenantAdmin(s). Store report in a SummaryReports table.
AppointmentReminderJob Every 30 minutes	Query confirmed appointments starting in 23.5–24.5 hours. Send reminder notification. Mark ReminderSent = true on the appointment to prevent duplicate sends.
FraudScoreRecalculationJob On-demand + daily 02:00 UTC	Re-evaluate FraudScore on all claims in Draft or PendingDocuments status. Useful when new historical data changes the median calculations. Must process in batches of 50 to avoid memory pressure.

10. Deliverables

10.1 Mandatory Submissions

Deliverable	Requirements
GitHub / GitLab Repository	Clean commit history showing progression. Minimum 15–20 commits. No single mega-commit. No commented-out code. Secrets never committed to repo.
README.md	Architecture diagram (ASCII or image). Prerequisites and how to run locally. Environment variable reference table. Design decisions for at least 2 significant choices (Architecture Decision Records).
docker-compose.yml	Spins up API + SQL Server + (optionally) Redis with a single command. All services start in the correct order.
EF Core Migrations	All tables created via dotnet ef database update from a clean state. Seed data: 1 tenant, 2 officers, 1 patient, 2 policy types, 5 sample claims in various states.
Test Suite	All tests pass with dotnet test. No skipped tests without an inline explanation. Coverage report generated (Coverlet).
Postman / .http File	Covers the full claim lifecycle: register → login → enroll policy → submit claim → officer review → settle.
Swagger / OpenAPI	All endpoints documented with XML comments, example request/response bodies, and Bearer auth scheme.

10.2 README Must Include

- Architecture diagram showing the 6-project solution and dependency arrows
- Honest completion matrix: which of the 6 complex scenarios are fully / partially implemented
- Database design rationale: why you chose specific indexes, the stored procedure approach
- Concurrency strategy: how you handle the three concurrency scenarios (claim approval, balance deduction, appointment booking)
- Instructions for running the full test suite and viewing the coverage report
- List of any bonus features implemented with brief description of approach